



ADDRESSING MODES

Name: Supriya Mishra

Roll no: 24229CMP008

Enrollment no: 476937

Course: B.Sc (hons) Computer Science

Year: 2nd (4th semester)

Subject: Computer Organisation
(CSCMJ44)





Content

- | | |
|---|--------------------------|
| • Introduction- What are addressing modes | • Register indirect mode |
| • Classification of addressing modes | • Base register mode |
| • Direct mode | • Indexed mode |
| • Indirect mode | • Implied mode |
| • Immediate mode | • Comparison table |
| • Relative address mode | • Conclusion |
| • Register mode | |
- 



INTRODUCTION

What are Addressing Modes?

Addressing modes are the methods used by a computer's CPU to specify the location of data (operands) that an instruction needs to execute.

In other words, addressing modes define how and where the processor accesses the required data for performing operations.

They determine whether the data is:

- Directly inside the instruction
- Stored in a register
- Located in memory
- Referred to through another address

Addressing modes are an essential part of instruction execution because they provide flexibility in accessing data efficiently.





Classification of addressing modes

- 1 Direct Addressing mode
 - 2 Indirect addressing mode
 - 3 Immediate addressing mode
 - 4 Relative address addressing mode
 - 5 Register addressing
 - 6 Register indirect addressing mode
 - 7 Base register addressing mode
 - 8 Indexed addressing mode
 - 9 Implied addressing mode
- 

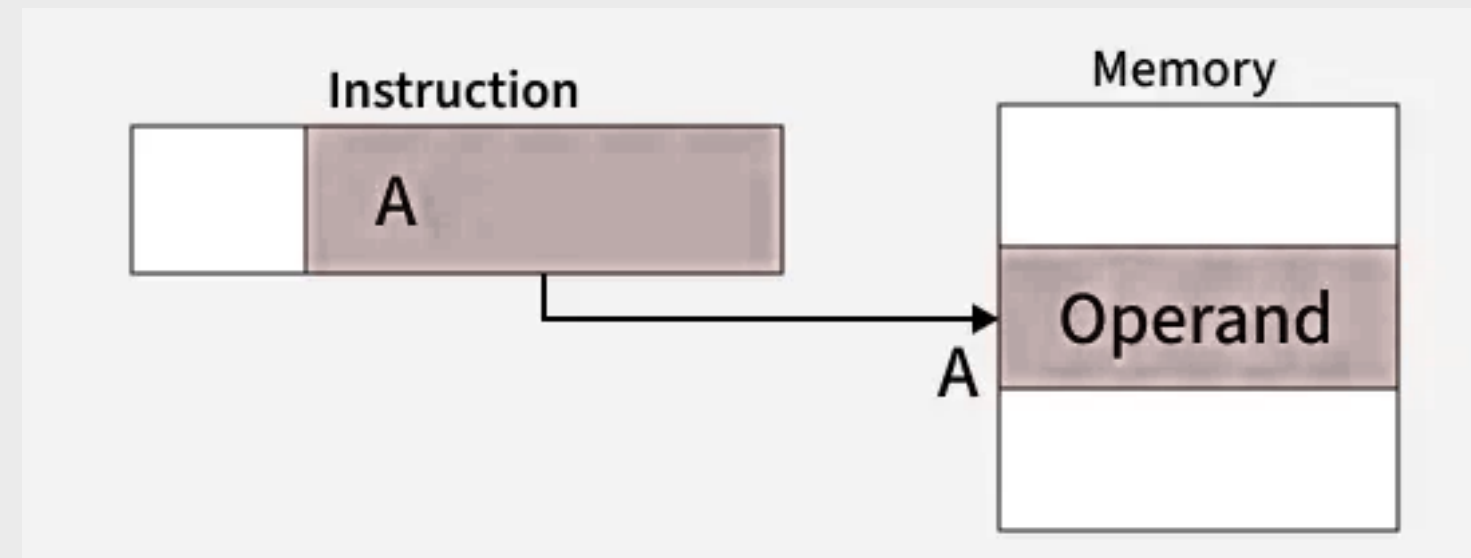
DIRECT MODE

Direct addressing mode is a method in which the address field of the instruction contains the actual memory address of the operand (data).

In this mode, the processor directly accesses the specified memory location to retrieve or store data, making it simpler than indirect methods. If the address field contains A, then the effective address (EA) is equal to A i.e, $EA=A$

Direct addressing mode is commonly used when data is stored in fixed memory locations.

Example: MOV A, [5000] Copies data from memory location 5000 to register A.



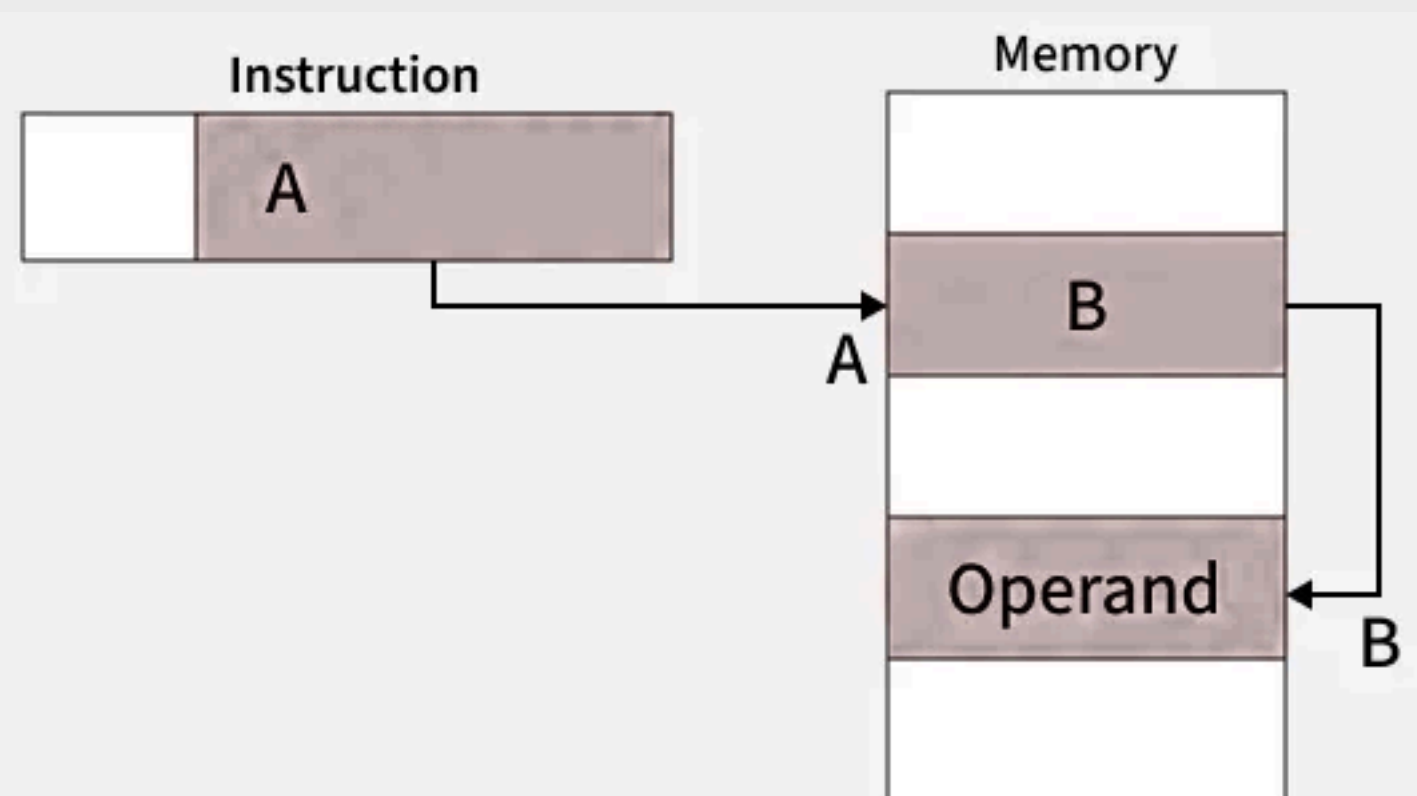
INDIRECT MODE

Indirect addressing mode is a method in which the address field of the instruction does not contain the actual operand address directly, but instead contains the address of another memory location that holds the effective address of the operand.

The effective address in these modes is obtained from the following computation:

Effective address = Address part of instruction + Content of CPU register

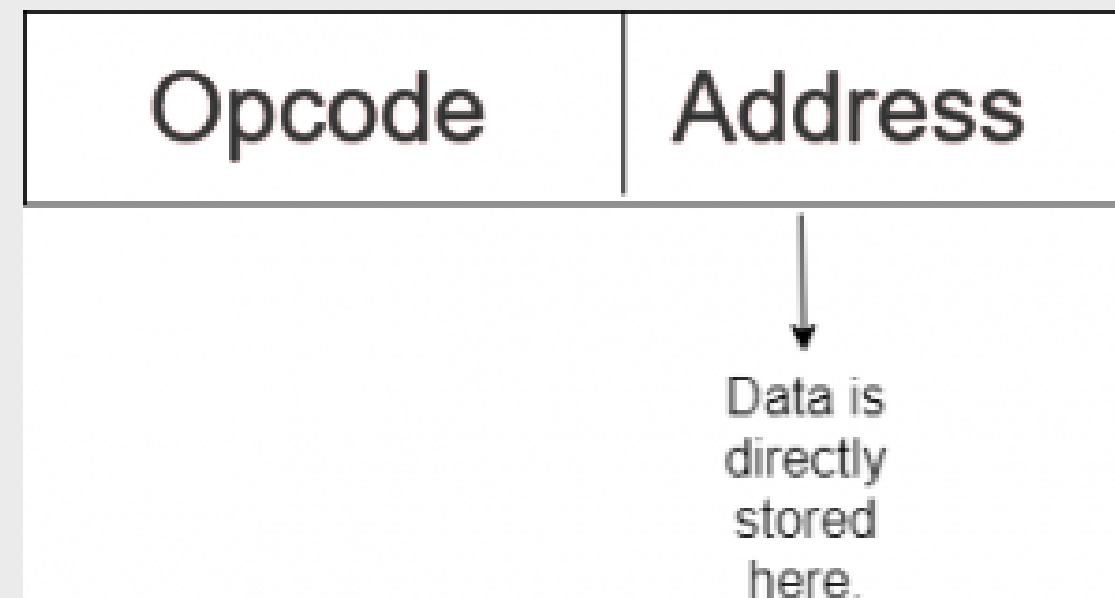
Example: LOAD R1, (A) Loads data from the memory location whose address is stored at memory location A.



IMMEDIATE MODE

Immediate addressing mode is a method in which the actual operand (data) is directly specified within the instruction itself, rather than referencing a memory location or register. Immediate mode instructions are useful for initializing registers to a constant value. In this mode, the instruction contains both the operation code and the data to be used, allowing the CPU to access the operand immediately during execution.

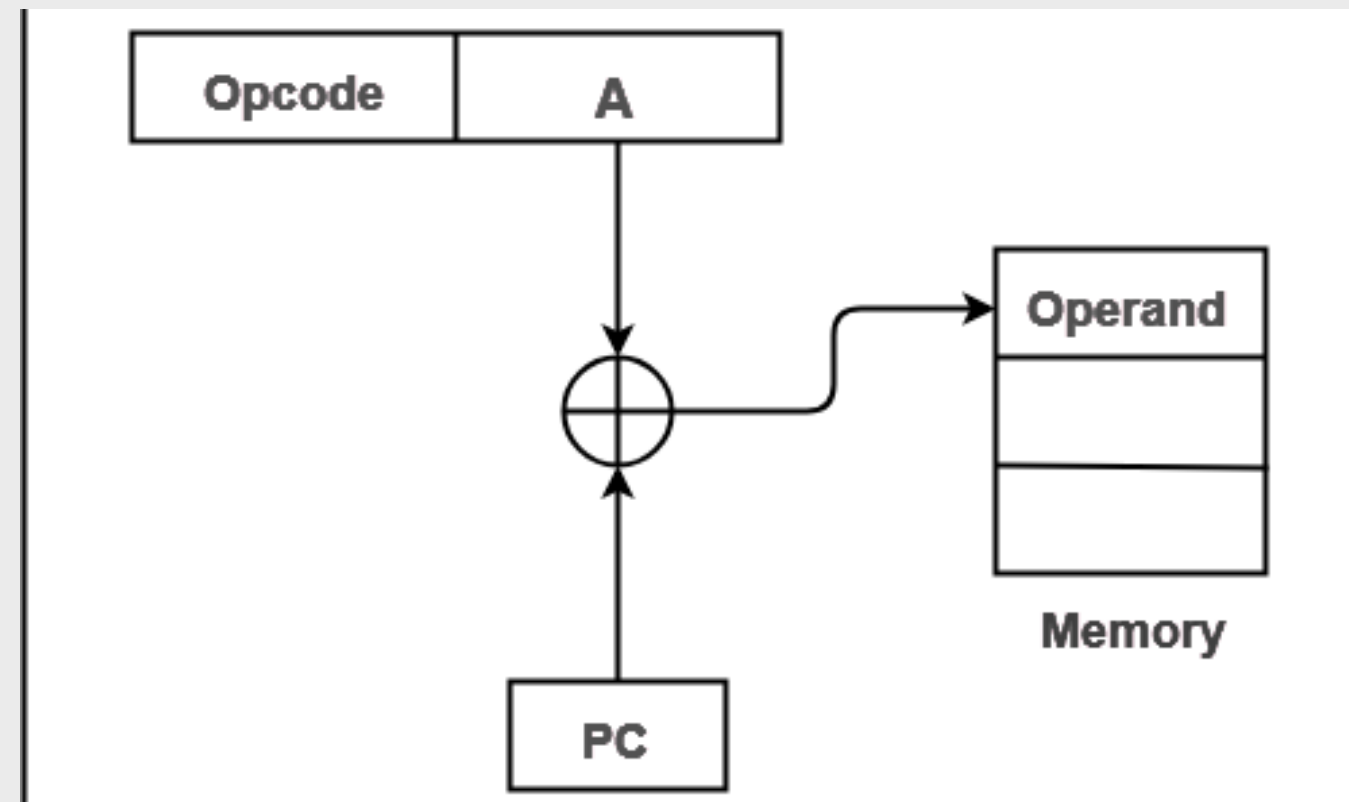
Example: MOV R1, #5 moves the value 5 into register R1, where #5 is the immediate value.



RELATIVE ADDRESS MODE

In relative address mode the content of the program counter is added to the address part of the instruction in order to obtain the effective address. The address part of the instruction is usually a signed number which can be either positive or negative. When this number is added to the content of the program counter, the result produces effective address whose position in memory is relative to the address of the next instruction.

Example: JMP +20 - The CPU jumps to the address obtained by adding 20 to the current Program Counter value.



REGISTER MODE

Register addressing mode is a method in which the operand (data) is stored directly in a CPU register, and the instruction specifies the register containing the required data.

In this mode, the processor accesses data from internal registers instead of memory locations, resulting in faster execution.

A k-bit field can specify any one of the 2^k registers.

Register addressing mode is commonly used in arithmetic operations, data transfer between registers, and temporary storage during program execution.

Example: MOV A, B Copies data from register B to register A.

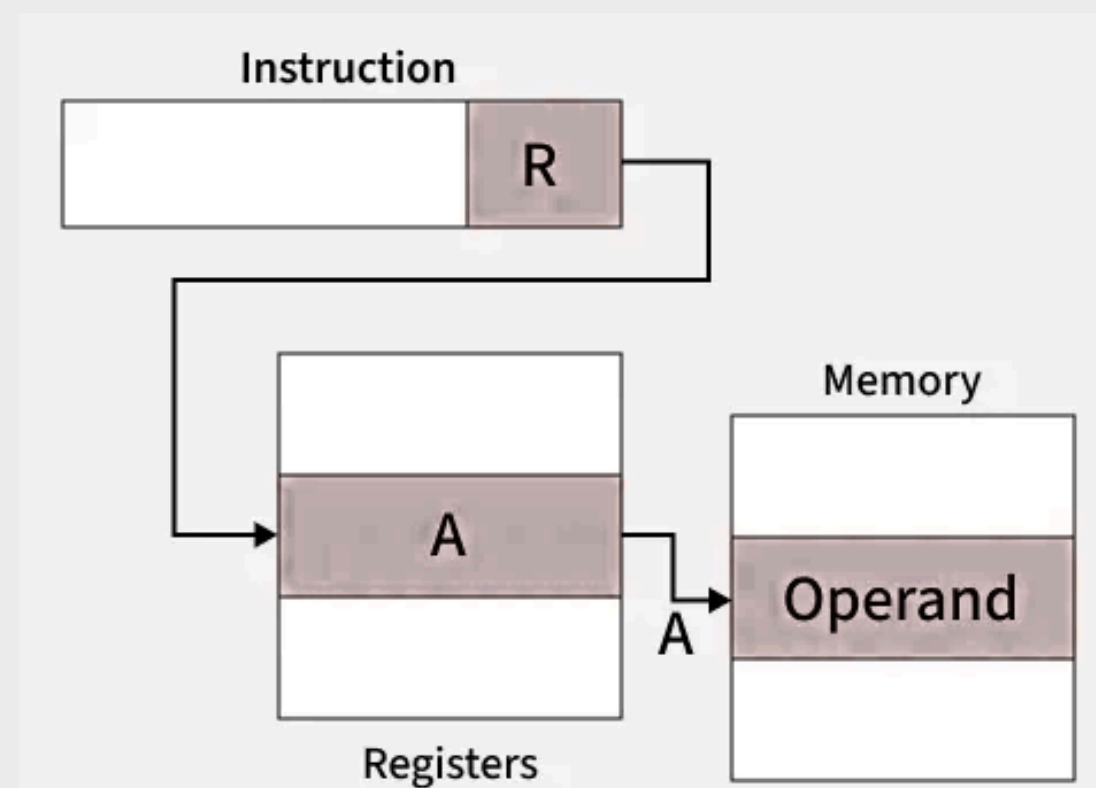


REGISTER INDIRECT MODE

Register indirect addressing mode is a method in which the instruction specifies a register that contains the memory address of the operand (data). In this mode the selected register contains the address of the operand rather than the operand itself.

The advantage of register in direct mode instruction is that the address field of the instruction uses fewer bits to select a register than would have been required to specify your memory address directly.

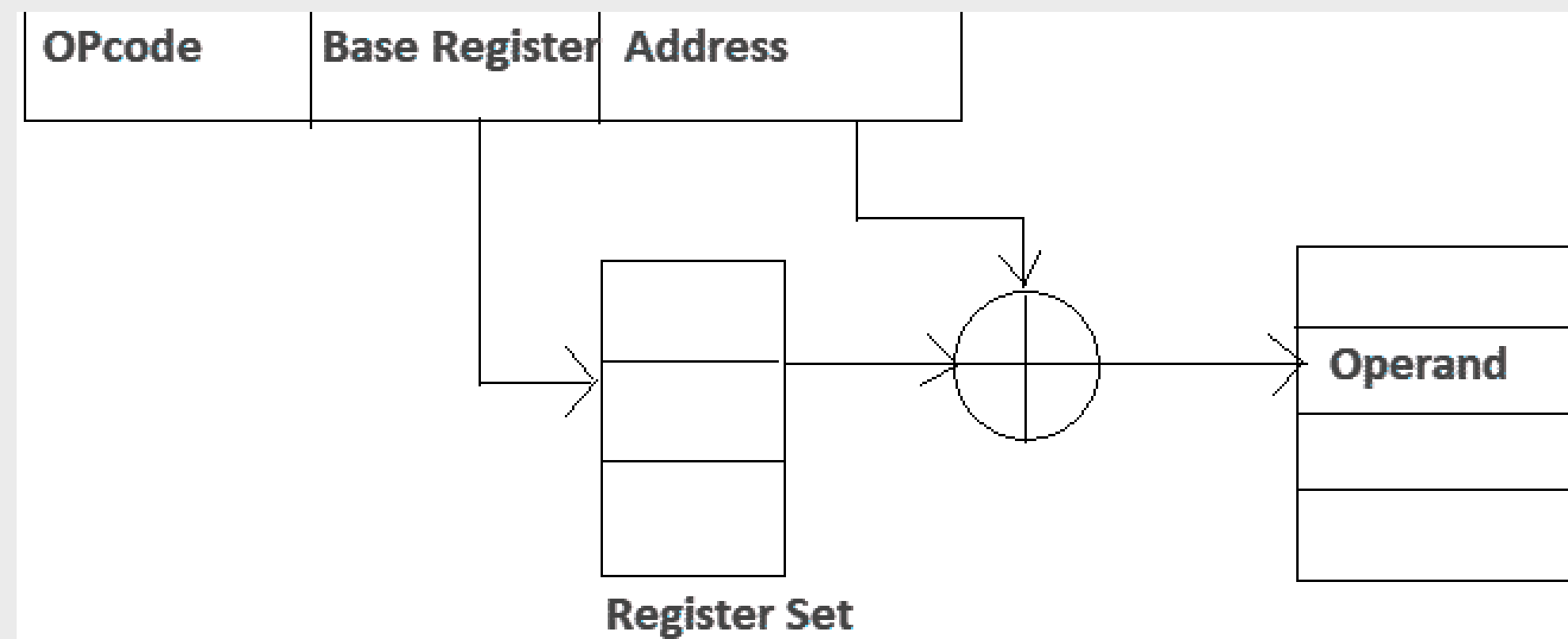
Example: LOAD R1, (R2). Loads data from the memory location whose address is stored in register R2.



BASE REGISTER MODE

Base register is the mode in which the content of the base register is added to the address part of the instruction to obtain the effective address. It is assumed to hold a base address and the address field of the instruction gives a displacement relative to the base address. It is primarily used for program relocation, managing data segments, and accessing data structures where the base address remains static while offsets change.

Example: MOV A, 20(R1) — The CPU accesses the data at the address obtained by adding displacement 20 to the base register R1.

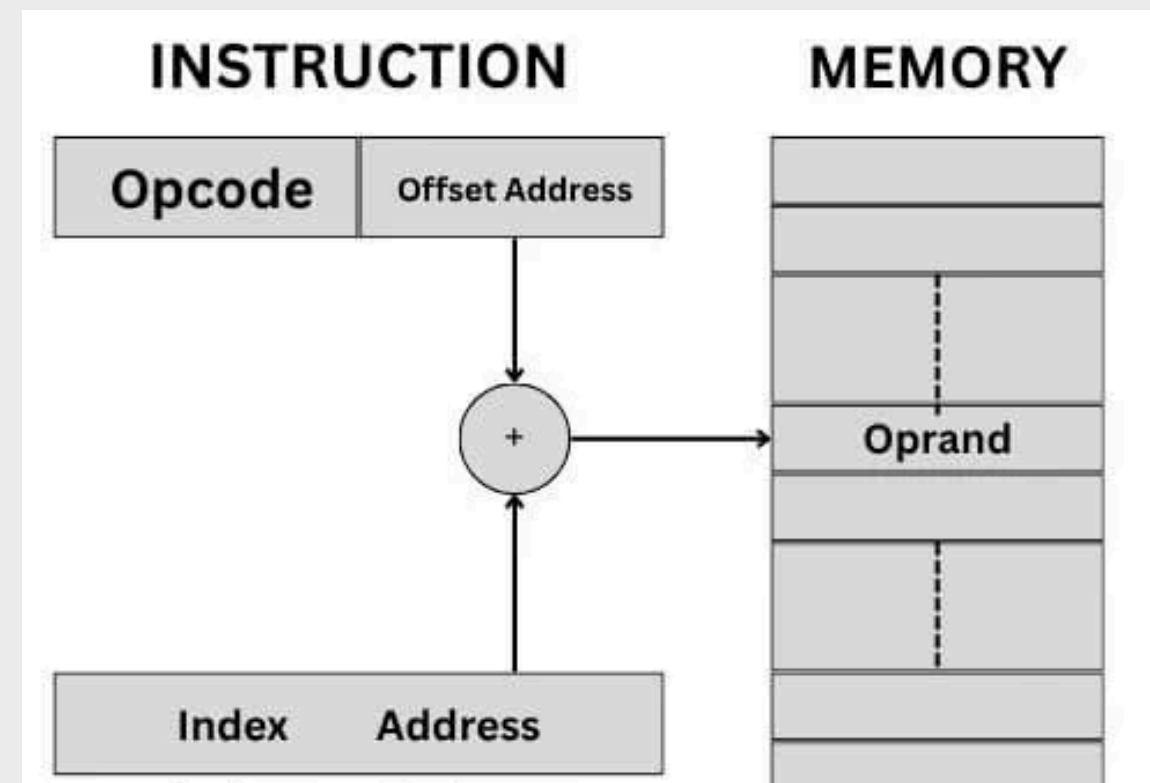


INDEXED MODE

Indexed addressing mode calculates an operand's effective memory address by adding a constant displacement (or base address) to the contents of a register, typically an index register (like SI or DI). It is used for accessing array elements, where the base address stays fixed and the register increments to iterate through data.

Indexed addressing mode is commonly used in array processing, string operations, table access, and looping operations.

Example: `MOV A, ARRAY[R1]` — The CPU accesses the data at the address obtained by adding the index register R1 to the base address of array.



IMPLIED MODE

Implied mode is the mode in which the operands are specified implicitly in definition of the instruction. The instruction “complement accumulator” is an implied-mode instruction because the operand in the accumulator register is implied in the definition of the instruction. Zero address instruction in a stack organized computer are implied mode instruction since the operands are implied to be on top of the stack.

Implied addressing mode is commonly used in accumulator-based operations, stack operations, and special-purpose instructions.

Example: CMA — The CPU automatically complements the accumulator without explicitly specifying the operand.



↓
Data is directly
stored here.

COMPARISON TABLE

Addressing mode	Operand location	Effective Address	Speed	Memory access
Immediate	Inside instruction	Operand itself	Very fast	No extra memory access
Direct	Memory location given directly	$EA = A$	Moderate	One memory access
Indirect	Address stored in memory	$EA = [A]$	Slower	Multiple memory access
Register	CPU register	Register itself	Very fast	No memory access
Register indirect	Memory address stored in register	$EA = (R)$	Fast	One memory access
Indexed	Base address+ index register	$EA = A + X$	Moderate	One memory access
Base register	Base register+ Displacement	$EA = B + A$	Moderate	One memory access
Implied	Operand implied in instruction	No Explicit EA	Very fast	No memory access
Relative addressing	Program counter +offset	$EA = PC + A$	Fast	One memory access



CONCLUSION

Addressing mode in action--

- **Logic & Efficiency:** Addressing modes provide the essential mapping between instruction bits and the Effective Address . They balance the trade-off between instruction compactness, execution speed, and the ability to access a vast memory space.
 - **Software & OS Support:** These modes are the physical backbone of high-level programming; Indirect and Indexed modes enable pointers and arrays, while PC-Relative addressing allows Operating Systems to load and run programs from any memory location.
 - **Hardware Optimization:** By utilizing Register and Immediate modes, CPUs bypass slow RAM access to perform high-speed arithmetic, while Auto-indexing automates stack management (Push/Pop) during critical function calls and system interrupts.
- 